ΜΑΘΗΜΑ 6 ΑΠΟ 15 · ΥΠΟΕΝΟΤΗΤΑ 2.3.Γ.3

# Συναρτήσεις

Γράφεις μια φορά — καλείς όσες φορές θες

Θεωρία σε 13 μικρά βήματα + 17 ασκήσεις στο εργαστήριο  Διάρκεια: 4 ώρες (3η–6η)  24 διαφάνειες

Προγραμματισμός Εφαρμογών Διαδικτύου & Παιχνιδιών (Python, Pygame) · Γ΄ εξάμηνο · Δευτέρα 9/11/2026

# Τι θα ξέρεις στο τέλος

1. Να **ορίζεις** δικές σου συναρτήσεις με **def** και να τις **καλείς** όσες φορές θες.
2. Να δίνεις **παραμέτρους** (και προεπιλεγμένες τιμές) και να παίρνεις πίσω αποτέλεσμα με **return**.
3. Να επιστρέφεις **δύο ή περισσότερες τιμές** μαζί, κάτι που η C δεν κάνει εύκολα.
4. Να ξεχωρίζεις **τοπικές** και **καθολικές** μεταβλητές (εμβέλεια).
5. Να χρησιμοποιείς **sorted, enumerate, zip, map, filter** και συναρτήσεις **lambda**.

### Πώς δουλεύουμε

Η θεωρία είναι χωρισμένη σε **13 μικρά βήματα**. Κάθε βήμα έχει πάνω δεξιά την ετικέτα **ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ** με τον αριθμό της άσκησης στο site.
**Διαβάζεις τη διαφάνεια → λύνεις την άσκηση → πας στο επόμενο βήμα.**

### Από το εύκολο στο δύσκολο

Ασκήσεις **1–6**: εύκολες, μία ιδέα τη φορά.
Ασκήσεις **7–13**: μέτριες, συνδυάζουν 2–3 βήματα.
Ασκήσεις **14–16**: δύσκολες, όλα μαζί. Η **17** είναι για το σπίτι.

**ΑΠΟ ΤΗ C** Στη C τις λέγατε **συναρτήσεις** με τύπο επιστροφής (int f(int x)). Στην Python: **def**, χωρίς τύπους.

# Συναρτήσεις χρησιμοποιείς ήδη!

**ΕΚΦΩΝΗΣΗ**

Πέντε γρήγορες ερωτήσεις, 5 λεπτά. Τι δίνει πίσω κάθε κλήση; Πρώτα σκέψου, μετά δες τη λύση.

**ΛΥΣΗ**

| ΚΛΉΣΗ | ΔΊΝΕΙ ΠΊΣΩ | ΤΙ ΕΊΝΑΙ |
|---|---|---|
| `len([4, 8, 15])` | `3` | ένας αριθμός |
| `max(7, 2, 9)` | `9` | ένας αριθμός |
| `sorted([3, 1, 2])` | `[1, 2, 3]` | μια νέα λίστα |
| `input("Όνομα: ")` | `ό,τι γράψεις` | πάντα κείμενο |
| `print("Γεια")` | `None` | μόνο εμφανίζει! |

### Όνομα + παρενθέσεις

Κάθε **len( )**, **max( )**, **print( )** είναι **κλήση συνάρτησης**: της δίνεις κάτι μέσα στις παρενθέσεις και κάνει μια δουλειά. Σήμερα φτιάχνουμε **δικές μας**.

**ΘΥΜΗΣΟΥ ΤΟ None** Το print **εμφανίζει**, δεν **δίνει πίσω** τίποτα. Αυτή η διαφορά είναι το βήμα 4.

# Ο ίδιος κώδικας, τρεις φορές

**ΜΑΘΗΜΑ 5: ΑΝΤΙΓΡΑΦΗ-ΕΠΙΚΟΛΛΗΣΗ**

```
while True:
    try:
        age = int(input("Ηλικία: "))
        break
    except ValueError:
        print("Δώσε αριθμό!")
while True:
    try:
        year = int(input("Έτος: "))
        break
    except ValueError:
        print("Δώσε αριθμό!")
# ... και ξανά για τον βαθμό ...
```

**ΣΗΜΕΡΑ: ΜΙΑ ΣΥΝΑΡΤΗΣΗ**

```
def read_int(prompt):
    while True:
        try:
            return int(input(prompt))
        except ValueError:
            print("Δώσε αριθμό!")

age = read_int("Ηλικία: ")
year = read_int("Έτος: ")
grade = read_int("Βαθμός: ")
```

## Τι στραβώνει

Θες να αλλάξεις το μήνυμα; Το αλλάζεις σε **τρία** σημεία — και κάπου θα το ξεχάσεις.

## Η λύση

Γράφεις τον κώδικα **μία φορά**, του δίνεις **όνομα** και τον καλείς όποτε τον χρειάζεσαι.

**ΤΟ ΣΧΕΔΙΟ** Βήματα **1–4**: φτιάχνω και καλώ. **5–8**: παράμετροι και εμβέλεια. **9–13**: έτοιμα εργαλεία της Python.

# Ορίζω και καλώ μια συνάρτηση

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Άσκηση 1**

ΟΡΙΣΜΟΣ ΚΑΙ ΚΛΗΣΗ

```
def hello():
    print("Γεια σου!")
    print("Καλή αρχή στο μάθημα.")

hello()          # 1η κλήση
hello()          # 2η κλήση
```

```
Γεια σου!
Καλή αρχή στο μάθημα.
Γεια σου!
Καλή αρχή στο μάθημα.
```

### def όνομα():

Η λέξη **def** (define = όρισε), το όνομα, **παρενθέσεις** και **:**. Οι γραμμές με εσοχή είναι το «σώμα» της.

### Ο ορισμός δεν τρέχει

Το def απλώς **θυμάται** τον κώδικα. Τρέχει μόνο όταν γράψεις **hello()** — αυτό λέγεται **κλήση**.

### Πρώτα ορίζω, μετά καλώ

Αν καλέσεις τη συνάρτηση **πάνω** από το def της → **NameError**.

ΟΝΟΜΑΤΑ Μικρά λατινικά με _ ανάμεσα: **read_int**, **show_menu**. Όχι ονόματα που υπάρχουν ήδη, όπως **sum** ή **list**.

# Δίνω τιμές στη συνάρτηση

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Άσκηση 2**

### ΜΙΑ ΠΑΡΑΜΕΤΡΟΣ

```
def greet(name):
    print(f"Γεια σου, {name}!")

greet("Άννα")
greet("Νίκος")
Γεια σου, Άννα!
Γεια σου, Νίκος!
```

### ΔΥΟ ΠΑΡΑΜΕΤΡΟΙ

```
def rectangle(w, h):
    print("Εμβαδό:", w * h)

rectangle(3, 5)
Εμβαδό: 15
```

### Παράμετρος

Μια μεταβλητή μέσα στις παρενθέσεις του def. Παίρνει τιμή **τη στιγμή της κλήσης**.

### Όρισμα

Η τιμή που δίνεις στην κλήση: στο **greet("Άννα")** το όρισμα είναι "Άννα" και μπαίνει στο **name**.

### Με τη σειρά

Στο **rectangle(3, 5)**: το 3 πάει στο **w**, το 5 στο **h**. Αν δώσεις λιγότερα → **TypeError**.

**ΑΠΟ ΤΗ C** Ίδια ιδέα με το void greet(char *name), αλλά **χωρίς τύπους**: δέχεται ό,τι του δώσεις.

# Η συνάρτηση δίνει πίσω αποτέλεσμα

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Άσκηση 3**

```
def square(x):
    return x * x

a = square(7)
print(a)
print(square(3) + square(4))
49
25
```

### ΠΩΣ ΤΡΕΧΕΙ ΤΟ square(3) + square(4)

| ΒΉΜΑ | ΤΙ ΓΊΝΕΤΑΙ |
|---|---|
| square(3) | x = 3 → δίνει πίσω **9** |
| square(4) | x = 4 → δίνει πίσω **16** |
| 9 + 16 | **25** → print |

### return τιμή

**Τελειώνει** τη συνάρτηση και στέλνει την τιμή **πίσω** εκεί που έγινε η κλήση. Η κλήση «γίνεται» αυτή η τιμή.

### Τι κάνω την τιμή;

Τη βάζω σε μεταβλητή (**a = square(7)**), την τυπώνω, ή τη χρησιμοποιώ σε πράξη — όπως με το **len()**.

### Μετά το return

Γραμμές κάτω από το return **δεν τρέχουν** ποτέ: η συνάρτηση έχει ήδη τελειώσει.

**ΧΩΡΙΣ return** Μια συνάρτηση χωρίς return δίνει πίσω **None** — όπως το print().

# Εμφανίζω ή δίνω πίσω;

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Άσκηση 4**

ΜΕ PRINT: ΤΟ ΒΛΕΠΩ, ΑΛΛΑ ΤΟ ΧΑΝΩ

```
def double_p(x):
    print(x * 2)

r = double_p(5)
10
print(r + 1)
TypeError: unsupported operand
type(s) for +: 'NoneType' and 'int'
```

ΜΕ RETURN: ΤΟ ΚΡΑΤΑΩ ΚΑΙ ΤΟ ΧΡΗΣΙΜΟΠΟΙΩ

```
def double_r(x):
    return x * 2

r = double_r(5)
print(r + 1)
11
```

### print μέσα στη συνάρτηση

Ο αριθμός πάει **στην οθόνη** και χάνεται. Το **r** γίνεται **None**, οπότε το r + 1 βγάζει λάθος.

### Ο κανόνας

Η συνάρτηση **υπολογίζει** και κάνει **return**. Το **print** το κάνει αυτός που την καλεί.

ΓΙΑΤΙ ΜΕΤΡΑΕΙ Με return το αποτέλεσμα **ξαναχρησιμοποιείται**: σε πράξη, σε if, σε λίστα, σε αρχείο.

# Τι θα εμφανιστεί;

```
def f(x):
    return x + 1

def g(x):
    print(x * 10)

print(f(2))
print(f(f(2)))
g(3)
print(g(3))
```

**ΕΚΦΩΝΗΣΗ**

Γράψε, με τη σειρά, κάθε γραμμή που εμφανίζεται. Προσοχή στις δύο τελευταίες!

**ΛΥΣΗ**

| ΕΝΤΟΛΉ | ΕΜΦΑΝΊΖΕΙ | ΓΙΑΤΊ |
|---|---|---|
| `print(f(2))` | `3` | 2 + 1 |
| `print(f(f(2)))` | `4` | f(3) → 4 |
| `g(3)` | `30` | το print του g |
| `print(g(3))` | `30` · `None` | **το g δεν έχει return** |

### Η ουσία

Στο **f(f(2))** τρέχει πρώτα το **μέσα** f(2) → 3, και μετά το f(3). Το **g** εμφανίζει μόνο του, αλλά δίνει πίσω **None**.

**ΔΟΚΙΜΑΣΕ** Τρέξε τον κώδικα στο «🧪 Ελεύθερο» του εργαστηρίου και έλεγξε τις απαντήσεις σου.

# Προεπιλεγμένες τιμές

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Άσκηση 5**

```
def power(base, exp=2):
    return base ** exp

print(power(5))       # exp = 2
print(power(2, 10))   # exp = 10
25
1024
```

ΤΟ ΕΙΧΕΣ ΗΔΗ ΔΕΙ: ΤΟ print

```
print("A", "B")
print("A", "B", sep="-")
print("A", end="!")
A B
A-B
A!
```

### exp=2

Αν στην κλήση **δεν** δώσεις τιμή για το exp, παίρνει το **2**. Αν δώσεις, παίρνει αυτή που έδωσες.

### Πάντα στο τέλος

Οι παράμετροι με προεπιλογή μπαίνουν **μετά** από τις υποχρεωτικές: **def f(a, b=1)** ✓, **def f(a=1, b)** ✗.

### Πού τη βλέπεις

Το **print** έχει **sep=" "** και **end="\n"** ως προεπιλογές. Γι' αυτό συνήθως δεν τα γράφεις.

ΠΟΤΕ ΤΗ ΒΑΖΩ Όταν μια τιμή είναι **σχεδόν πάντα** ίδια, αλλά καμιά φορά θες να την αλλάξεις.

# Ονομαστικά ορίσματα

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Άσκηση 6**

```
def ticket(name, seat, price=10):
    print(f"{name} · θέση {seat} · {price}€")

ticket("Άννα", 12)
ticket(seat=5, name="Νίκος")
ticket("Ελένη", 7, price=7.5)
Άννα · θέση 12 · 10€
Νίκος · θέση 5 · 10€
Ελένη · θέση 7 · 7.5€
```

### όνομα=τιμή

Στην κλήση γράφεις **ποια** παράμετρος παίρνει **ποια** τιμή. Τότε η **σειρά δεν μετράει**.

### Πιο καθαρό

Το **ticket("Ελένη", 7, price=7.5)** διαβάζεται καλύτερα από ένα σκέτο ticket("Ελένη", 7, 7.5).

### Ο κανόνας

Πρώτα τα **θέσης**, μετά τα **ονομαστικά**: ticket(seat=5, "Νίκος") → **SyntaxError**.

ΤΟ ΕΧΕΙΣ ΔΕΙ **sorted(grades, reverse=True)**: το reverse είναι ονομαστικό όρισμα με προεπιλογή False.

# Επιστρέφω δύο ή περισσότερες τιμές

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Άσκηση 7**

```
def stats(nums):
    avg = sum(nums) / len(nums)
    return min(nums), max(nums), avg

lo, hi, avg = stats([12, 8, 17, 15])
print("Μικρότερος:", lo)
print("Μεγαλύτερος:", hi)
print(f"Μέσος όρος: {avg:.2f}")
Μικρότερος: 8
Μεγαλύτερος: 17
Μέσος όρος: 13.00
```

### return a, b, c

Γράφεις τις τιμές με **κόμμα**. Η Python τις «πακετάρει» σε μια **πλειάδα** (Μάθημα 3).

### lo, hi, avg = ...

Τις **μοιράζεις** σε μεταβλητές, όπως το **x, y = point**. Πρέπει να είναι **όσες** επιστρέφει.

### Η C δεν το κάνει

Στη C θα χρειαζόσουν **δείκτες** ή struct. Στην Python είναι μία γραμμή.

**ΔΟΚΙΜΑΣΕ** Γράψε **r = stats([1, 2, 3])** και **print(r)**: θα δεις την πλειάδα **(1, 3, 2.0)**.

# Τοπικές και καθολικές μεταβλητές

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Άσκηση 8**

ΤΟΠΙΚΗ

```
def area(r):
    pi = 3.14           # τοπική: ζει μόνο εδώ μέσα
    return pi * r * r

print(area(2))
print(pi)
12.56
NameError: name 'pi' is not defined
```

ΚΑΘΟΛΙΚΗ ΜΕ global

```
score = 0               # καθολική

def add_point():
    global score        # «θα αλλάξω την καθολική»
    score = score + 10

add_point()
add_point()
print(score)
20
```

### Τοπική

Φτιάχνεται **μέσα** στη συνάρτηση (και οι παράμετροι). **Σβήνει** όταν τελειώσει η κλήση. Απ' έξω δεν φαίνεται.

### Καθολική

Φτιάχνεται **έξω** από κάθε def. Η συνάρτηση μπορεί να τη **διαβάσει**.

### global

Για να την **αλλάξεις** μέσα σε συνάρτηση, γράψε πρώτα **global όνομα**. Αλλιώς φτιάχνεται νέα τοπική με το ίδιο όνομα.

### Με μέτρο

Το global κάνει τον κώδικα δύσκολο να τον παρακολουθήσεις. Προτίμησε **παράμετρο + return**.

ΓΙΑΤΙ ΕΙΝΑΙ ΚΑΛΟ Κάθε συνάρτηση έχει τις **δικές της** μεταβλητές: ένα i μέσα της δεν χαλάει το i του υπόλοιπου προγράμματος.

# Ποιο x είναι ποιο;

```
x = 5

def a():
    x = 100
    print("a:", x)

def b():
    print("b:", x)

def c():
    global x
    x = x + 1

a()
b()
c()
print("τέλος:", x)
```

**ΕΚΦΩΝΗΣΗ**

Γράψε τι εμφανίζει κάθε γραμμή. Ποια συνάρτηση αλλάζει **πραγματικά** το x;

**ΛΥΣΗ**

| ΓΡΑΜΜΉ | ΕΜΦΑΝΊΖΕΙ | ΓΙΑΤΊ |
|---|---|---|
| `a()` | `a: 100` | νέα **τοπική** x |
| `b()` | `b: 5` | διαβάζει την καθολική |
| `c()` | — | **global**: την αλλάζει |
| `print` | `τέλος: 6` | μόνο το c την άλλαξε |

### Η παγίδα

Το **x = 100** μέσα στο a **δεν** αγγίζει το καθολικό x: φτιάχνει ένα **άλλο** x που σβήνει μόλις τελειώσει το a.

**ΤΕΧΝΙΚΗ** Για κάθε συνάρτηση ρώτα: «**γράφω** κάπου x = …; Αν ναι, χωρίς global είναι **τοπική**».

# Όσα ορίσματα θέλεις

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Ασκήσεις 9 · 10**

### *ARGS: ΟΣΟΙ ΑΡΙΘΜΟΙ ΘΕΣ · ΑΣΚΗΣΗ 9

```
def total(*nums):
    s = 0
    for n in nums:
        s = s + n
    return s

print(total(4, 6))
print(total(1, 2, 3, 4))
print(total())
10
10
0
```

### **KWARGS: ΟΣΑ ΟΝΟΜΑΣΤΙΚΑ ΘΕΣ · ΑΣΚΗΣΗ 10

```
def profile(**info):
    for key, value in info.items():
        print(f"{key}: {value}")

profile(name="Άννα", age=19, city="Γλυφάδα")
name: Άννα
age: 19
city: Γλυφάδα
```

***nums**

Μαζεύει όλα τα ορίσματα θέσης σε μια **πλειάδα**: (1, 2, 3, 4).

****info**

Μαζεύει όλα τα ονομαστικά σε ένα **λεξικό**: {'name': 'Άννα', ...}.

ΤΟ ΕΧΕΙΣ ΔΕΙ Το **print** δέχεται όσα ορίσματα θες — γιατί έχει *args. Τα ονόματα args/kwargs είναι συνήθεια· το * είναι που μετράει.

# sorted · enumerate · zip

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Άσκηση 11**

## ENUMERATE: ΘΕΣΗ ΚΑΙ ΣΤΟΙΧΕΙΟ

```
names = ["Νίκος", "Άννα", "Ελένη"]
grades = [9, 17, 15]

for i, name in enumerate(names, 1):
    print(i, name)
1 Νίκος
2 Άννα
3 Ελένη
```

## ZIP: ΔΥΟ ΛΙΣΤΕΣ ΔΙΠΛΑ-ΔΙΠΛΑ

```
for name, g in zip(names, grades):
    print(f"{name}: {g}")
Νίκος: 9
Άννα: 17
Ελένη: 15
```

### enumerate(λίστα, 1)

Δίνει ζευγάρια **(αριθμός, στοιχείο)**. Το 1 λέει «ξεκίνα να μετράς από το 1». Τέλος το range(len(...))!

### zip(α, β)

Δίνει ζευγάρια **(α[0], β[0])**, **(α[1], β[1])**, ... Σταματά στη **μικρότερη** λίστα.

### sorted(...)

Επιστρέφει **νέα** ταξινομημένη λίστα — η αρχική μένει ίδια. Με **reverse=True** ανάποδα, με **key=** «κατά τι» (βήμα 12).

### dict(zip(...))

Δύο λίστες γίνονται λεξικό: **dict(zip(names, grades))** (Μάθημα 4).

**ΘΥΜΗΣΟΥ** Στο **for i, name in ...** μοιράζεις κάθε ζευγάρι σε δύο μεταβλητές, όπως στο **.items()** των λεξικών.

# Μια συνάρτηση σε κάθε στοιχείο

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Άσκηση 12**

## MAP: ΕΦΑΡΜΟΣΕ ΣΕ ΟΛΑ

```
def with_vat(p):
    return round(p * 1.24, 2)

prices = [2.5, 10, 4.2]
print(list(map(with_vat, prices)))
[3.1, 12.4, 5.21]
```

## FILTER: ΚΡΑΤΑ ΟΣΑ ΠΕΡΝΑΝΕ

```
def is_expensive(p):
    return p > 4

print(list(filter(is_expensive, prices)))
[10, 4.2]
```

## ΑΚΟΜΑ ΔΥΟ ΧΡΗΣΙΜΕΣ

| ΚΛΗΣΗ | ΔΙΝΕΙ | ΤΙ ΚΑΝΕΙ |
|---|---|---|
| `abs(-7)` | 7 | απόλυτη τιμή |
| `round(3.14159, 2)` | 3.14 | στρογγυλεύει |
| `round(2.5)` | 2 | στο **ζυγό**! |

### map(συνάρτηση, λίστα)

Καλεί τη συνάρτηση για **κάθε** στοιχείο. Δίνεις το **όνομα** χωρίς παρενθέσεις: with_vat, όχι with_vat().

### list(...)

Τα map και filter δίνουν τα αποτελέσματα «ένα-ένα». Με **list()** τα μαζεύεις σε λίστα για να τα τυπώσεις.

ΙΔΙΟ ΜΕ Το map είναι ένα for με **append(f(x))**· το filter ένα for με **if** και **append(x)** (Μάθημα 3).

# Μικρές συναρτήσεις χωρίς όνομα

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Άσκηση 13**

### LAMBDA ΠΑΡΑΜΕΤΡΟΣ: ΑΠΟΤΕΛΕΣΜΑ

```
double = lambda x: x * 2
print(double(21))
42

print(list(map(lambda p: p * 1.24, [10, 20])))
[12.4, 24.8]
```

### ΤΟ ΠΙΟ ΣΥΧΝΟ: ΤΑΞΙΝΟΜΗΣΗ ΜΕ key=

```
students = [{"name": "Νίκος", "grade": 9},
            {"name": "Άννα", "grade": 17},
            {"name": "Ελένη", "grade": 15}]

best = sorted(students, key=lambda s: s["grade"], reverse=True)
for s in best:
    print(s["name"], s["grade"])
Άννα 17
Ελένη 15
Νίκος 9
```

### lambda x: x * 2

Ίδιο με **def f(x): return x * 2**, σε μία γραμμή. Μία μόνο **έκφραση**, χωρίς return.

### key=lambda s: s["grade"]

Λέει στο sorted **κατά τι** να ταξινομήσει: για κάθε λεξικό, κοίτα τον **βαθμό**.

### Πότε lambda;

Όταν τη χρειάζεσαι **μία φορά**, μέσα σε sorted, map, filter, max, min. Για ό,τι μεγαλύτερο: **def**.

**ΔΟΚΙΜΑΣΕ** **max(students, key=lambda s: s["grade"])** δίνει κατευθείαν τον καλύτερο μαθητή.

# Γράφω τι κάνει η συνάρτηση

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Άσκηση 14**

```
def is_prime(n):
    """Επιστρέφει True αν ο n είναι πρώτος."""
    if n < 2:
        return False
    for d in range(2, int(n ** 0.5) + 1):
        if n % d == 0:
            return False      # βρήκα διαιρέτη: τέλος
    return True

help(is_prime)
is_prime(n)
    Επιστρέφει True αν ο n είναι πρώτος.
print(is_prime(17), is_prime(91))
True False
```

### """…"""

Το **docstring**: κείμενο σε τριπλά εισαγωγικά, **αμέσως κάτω** από το def. Λέει **τι** κάνει η συνάρτηση, όχι πώς.

### help(όνομα)

Εμφανίζει το docstring. Το ίδιο βλέπεις στο **VS Code** όταν περνάς το ποντίκι πάνω από την κλήση.

### return μέσα στο for

Μόλις βρει διαιρέτη, το return **σταματά** και τον βρόχο και τη συνάρτηση. Δεν χρειάζεται break.

### Καλή συνάρτηση

Κάνει **ένα πράγμα**, έχει **καθαρό όνομα** και **docstring**, και δίνει το αποτέλεσμα με **return**.

**ΓΙΑΤΙ ΕΩΣ √n** Αν ο n έχει διαιρέτη, έχει κι έναν **≤ √n**. Για το 1.000.000 ψάχνεις 1.000 αριθμούς αντί για 999.998.

# Οι 17 ασκήσεις, βήμα-βήμα

| # | ΆΣΚΗΣΗ | ΕΠΊΠΕΔΟ | ΔΙΑΦΆΝΕΙΑ |
|---|---|---|---|
| 1 | Η πρώτη μου συνάρτηση | Εύκολο | 5 |
| 2 | Χαιρετισμός με παράμετρο | Εύκολο | 6 |
| 3 | Τετράγωνο με return | Εύκολο | 7 |
| 4 | Ο μεγαλύτερος από δύο | Εύκολο | 8 |
| 5 | Δύναμη με προεπιλογή | Εύκολο | 10 |
| 6 | Εισιτήρια | Εύκολο | 11 |
| 7 | Στατιστικά: 3 τιμές πίσω | Μέτριο | 12 |
| 8 | Σκορ με global | Μέτριο | 13 |
| 9 | Άθροισμα με *args | Μέτριο | 15 |

| # | ΆΣΚΗΣΗ | ΕΠΊΠΕΔΟ | ΔΙΑΦΆΝΕΙΑ |
|---|---|---|---|
| 10 | Προφίλ με **kwargs | Μέτριο | 15 |
| 11 | Πίνακας βαθμών | Μέτριο | 16 |
| 12 | ΦΠΑ με map και filter | Μέτριο | 17 |
| 13 | Ταξινόμηση με lambda | Μέτριο | 18 |
| 14 | Πρώτοι αριθμοί | Δύσκολο | 19, 21 |
| 15 | Ρώτα ξανά, ως συνάρτηση | Δύσκολο | 4, 22 |
| 16 | Η αναφορά της τάξης | Ο στόχος | 18, 23 |
| 17 | Αριθμομηχανή | Για το σπίτι | 7, 10 |

**ΣΤΟ SITE** Εργαστήριο → **Μάθημα 6**. Κάθε άσκηση έχει το «📖 Στις διαφάνειες» που ανοίγει κατευθείαν τη σωστή διαφάνεια.

# Πρώτοι αριθμοί ως το n

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Άσκηση 14**

**ΕΚΦΩΝΗΣΗ**

Γράψε τη συνάρτηση **is_prime(n)** που επιστρέφει True ή False. Μετά διάβασε ένα n και, **καλώντας** την is_prime, εμφάνισε όλους τους πρώτους ως το n και πόσοι είναι.

**ΛΥΣΗ · ΔΟΚΙΜΑΣΕ ΠΡΩΤΑ ΜΟΝΟΣ/Η ΣΟΥ**

```
def is_prime(n):
    """Επιστρέφει True αν ο n είναι πρώτος."""
    if n < 2:
        return False
    for d in range(2, int(n ** 0.5) + 1):
        if n % d == 0:
            return False
    return True

n = int(input("Έως: "))
primes = []
for k in range(2, n + 1):
    if is_prime(k):
        primes.append(k)
print(primes)
print("Πλήθος:", len(primes))
```

```
Έως: 30
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
Πλήθος: 10
```

### if is_prime(k):

Η συνάρτηση δίνει πίσω **True/False**, οπότε μπαίνει **κατευθείαν** στο if. Δεν χρειάζεται == True.

**ΑΠΟ ΠΟΙΑ ΒΗΜΑΤΑ** **3** return · **13** return μέσα στο for · Μάθημα 2: **πρώτοι αριθμοί** · Μάθημα 3: **append**.

# Το «ρώτα ξανά» ως συνάρτηση

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Άσκηση 15**

**ΕΚΦΩΝΗΣΗ**

Γράψε **read_int(prompt, lo, hi)**: ρωτά ξανά και ξανά μέχρι ο χρήστης να δώσει **ακέραιο** από lo έως hi, και τον **επιστρέφει**. Με αυτή διάβασε ηλικία (15–99) και μήνα (1–12).

**ΛΥΣΗ · ΔΟΚΙΜΑΣΕ ΠΡΩΤΑ ΜΟΝΟΣ/Η ΣΟΥ**

```
def read_int(prompt, lo, hi):
    """Ζητά ακέραιο από lo έως hi μέχρι να δοθεί σωστός."""
    while True:
        try:
            n = int(input(prompt))
        except ValueError:
            print("Δώσε ακέραιο!")
            continue
        if lo <= n <= hi:
            return n
        print(f"Από {lo} έως {hi}!")

age = read_int("Ηλικία: ", 15, 99)
month = read_int("Μήνας: ", 1, 12)
print(f"Ηλικία {age}, μήνας {month}")
```

```
Ηλικία: δεκαεννιά
Δώσε ακέραιο!
Ηλικία: 19
Μήνας: 13
Από 1 έως 12!
Μήνας: 11
Ηλικία 19, μήνας 11
```

### return μέσα στο while

Βγάζει **κατευθείαν** από βρόχο και συνάρτηση, με τον σωστό αριθμό.

### Από ποια βήματα;

**2** παράμετροι · **3** return · Μάθημα 5: **try / except**, «ρώτα ξανά»

**ΚΡΑΤΑ ΤΗ** Αυτή η συνάρτηση θα σου χρειαστεί σε **κάθε** πρόγραμμα με μενού. Στο Μάθημα 7 θα τη βάλουμε σε δικό της αρχείο.

# Η αναφορά της τάξης

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Άσκηση 16**

ΛΥΣΗ · ΔΟΚΙΜΑΣΕ ΠΡΩΤΑ ΜΟΝΟΣ/Η ΣΟΥ

```
students = [
    {"name": "Άννα", "grades": [17, 15, 18]},
    {"name": "Νίκος", "grades": [9, 11, 8]},
    {"name": "Ελένη", "grades": [14, 16, 15]},
    {"name": "Γιώργος", "grades": [12, 7, 10]},
]

def average(st):
    """Ο μέσος όρος ενός μαθητή."""
    return sum(st["grades"]) / len(st["grades"])

def ranking(students):
    """Οι μαθητές από τον καλύτερο στον χειρότερο."""
    return sorted(students, key=average, reverse=True)

def report(students, limit=10):
    for i, st in enumerate(ranking(students), 1):
        mark = "✓" if average(st) >= limit else "✗"
        print(f"{i}. {st['name']:8} {average(st):5.2f} {mark}")

report(students)
```

```
1. Άννα     16.67 ✓
2. Ελένη    15.00 ✓
3. Γιώργος   9.67 ✗
4. Νίκος     9.33 ✗
```

### key=average

Το key θέλει **συνάρτηση**: δίνεις τη δική σου, **χωρίς** παρενθέσεις. Ίδιο με key=lambda st: average(st).

### Τρεις μικρές συναρτήσεις

Η καθεμία κάνει **ένα** πράγμα. Η report **καλεί** τις άλλες δύο.

### limit=10

Προεπιλογή (βήμα 5): **report(students, 15)** δείχνει ποιοι έχουν από 15 και πάνω.

**ΔΟΚΙΜΑΣΕ** Πρόσθεσε μια μαθήτρια στη λίστα: η αναφορά βγαίνει σωστή **χωρίς** να αλλάξεις καμία συνάρτηση.

# Τι κρατάμε

| ΘΈΛΩ ΝΑ… | ΓΡΆΦΩ |
|---|---|
| ορίσω συνάρτηση | `def όνομα(a, b):` |
| δώσω πίσω αποτέλεσμα | `return τιμή` |
| δώσω πολλά αποτελέσματα | `return a, b → x, y = f()` |
| βάλω προεπιλογή | `def f(a, b=2):` |
| καλέσω με ονόματα | `f(b=5, a=1)` |
| αλλάξω καθολική | `global όνομα` |
| δεχτώ όσα ορίσματα θέλω | `def f(*args, **kwargs):` |
| αριθμήσω / ενώσω λίστες | `enumerate(λ, 1) · zip(α, β)` |
| εφαρμόσω / φιλτράρω | `map(f, λ) · filter(f, λ)` |
| ταξινομήσω «κατά τι» | `sorted(λ, key=lambda x: …)` |

### Γράφεις μια φορά

Η συνάρτηση δίνει **όνομα** σε ένα κομμάτι κώδικα. Το διορθώνεις σε **ένα** σημείο και το καλείς παντού.

### Επόμενη φορά · Δευ 16/11

**Αναδρομή** (συνάρτηση που καλεί τον εαυτό της), δικά μας **modules**, και γραφήματα με **NumPy** και **Matplotlib**.

**ΓΙΑ ΤΟ ΣΠΙΤΙ** Άσκηση **17** στο εργαστήριο: αριθμομηχανή με μία συνάρτηση calc(a, op, b) (βήματα 3 και 5).